Contents

CS683 PA2 - Task 3: Incorporating SPP with Dynamic I-RRIP **1**
1. Headline 1
2. Prefetch accuracy and coverage (the PDF deliverable) 1
3. Per-trace IPC and L2C load MPKI 1
4. Why handle the prefetch packet this way (rationale) 1

CS683 PA2 - Task 3: Incorporating SPP with Dynamic I-RRIP

Every figure is recomputed from `results/task3/*.txt` at build time.

1. Headline

Baseline LRU, 25M warmup + 25M simulation, GCC 7.5.0, three traces. Speedup is the geomean of per-trace IPC ratios; MPKI is the arithmetic mean.

policy	geomean speedup	mean L1D MPKI	mean L2C MPKI	mean L2 instr	mean L2 data
LRU (baseline)	1.0000	9.2064	16.3355	10.1050	6.2305
SPP alone	1.0149	8.4431	15.6627	10.1670	5.4957
dynamic I-RRIP alone	1.0068	9.2089	14.5693	7.2543	7.3150
SPP + dynamic I-RRIP	1.0218	8.4480	13.8400	7.2537	6.5863

Task 3 bar: combined **1.0218** vs SPP-alone **1.0149** vs dynamic-I-RRIP-alone **1.0068** -> combined beats both: **YES**.

2. Prefetch accuracy and coverage (the PDF deliverable)

Accuracy = useful / lines actually fetched (ChampSim's own definition, reused from Task 1); coverage = reduction in L1D load misses vs LRU.

policy	trace1 acc%	trace2 acc%	trace3 acc%	trace1 cov%	trace2 cov%	trace3 cov%
LRU (baseline)	0.00	0.00	0.00	0.00	0.00	0.00
SPP alone	56.69	54.61	47.71	10.51	3.99	18.45
dynamic I-RRIP alone	0.00	0.00	0.00	-0.00	-0.05	-0.00
SPP + dynamic I-RRIP	56.62	54.68	47.71	10.49	3.91	18.39

3. Per-trace IPC and L2C load MPKI

trace	LRU (baseline)	SPP alone	dynamic I-RRIP alone	SPP + dynamic I-RRIP
trace1 IPC	0.23234	0.23454	0.23210	0.23429
trace1 L2C MPKI	17.6956	16.8508	17.7756	16.9036
trace2 IPC	0.25116	0.25418	0.25692	0.26015
trace2 L2C MPKI	27.3568	26.9366	21.9398	21.3890
trace3 IPC	0.38206	0.39092	0.38159	0.39027
trace3 L2C MPKI	3.9542	3.2006	3.9924	3.2275

4. Why handle the prefetch packet this way (rationale)

A `type == PREFETCH` fill is a prediction, not a demand. Our Task 1 data shows SPP is right only ~48-57%% of the time, so standalone SPP+LRU – which inserts every prefetch at MRU – lets the wrong ~half pollute the L2 and evict demand lines. The combined policy instead inserts a prefetch at a distant RRPV so a wrong prediction ages out fast, and promotes it to a demand line only when a real demand access hits it (proven useful). Instruction protection is untouched, and prefetch fills never vote in the duel – so the two

mechanisms are orthogonal: SPP cuts data-miss latency, I-RRIP protects instruction reuse, and prefetch-aware insertion stops SPP from polluting what I-RRIP protects. The insertion depth was chosen by sweep (scripts/sweep_task3.py), not assumed; the unaware ablation in that sweep isolates awareness as the cause.

See docs/TASK3_DESIGN_DECISIONS.md for the full argument and the sweep evidence.